모델 저장과 불러오기

학습한 모델, 어떻게 보관할까?

pkl 파일 이해하기



model.pkl

왜 저장해야 할까?

Colab을 끄면, 학습한 모델도 사라진다

학습

리뷰 수만 건으로 모델을 학습 (몇 분~ 몇 시간)

런타임 종료

모델은 컴퓨터 메모리에만 있었기 때문에 함께 지워짐

처음부터 다시

쓸 때마다 다시 학습해야 함

해결: 학습이 끝난 모델을 파일로 저장해 두고, 필요할 때 불러와 바로 쓴다

pkl = pickle (피클, 절임)

만들어 둔 것을 절여 두었다가 꺼내 쓴다

요리

① 만들기

모델 학습

`model.fit()`

병에 담아 보관

② 저장

파일로 저장

`joblib.dump()`

꺼내 먹기

③ 불러오기

다시 학습 없이 예측

`joblib.load()`

→ 한 번 오래 학습하고, 이후에는 몇 초 만에 불러와 쓴다

pkl에 담기는 것

파이썬 안의 무엇이든 통째로

학습한 모델

로지스틱 회귀, KNN 등

단어집

TF-IDF, Tokenizer

표 · 리스트

추천용 상품 목록 등

설정값

기준값, 이름표 등





○○○.pkl

여러 개를 한 파일에
묶어 담을 수도 있음

익숙한 파일과 비교

csv는 데이터, pkl은 파이썬 물건

구분	csv	pkl
담는 것	표 데이터만	모델 · 단어집 · 리스트 등 무엇이든
여는 방법	메모장 · 엑셀 · 파이썬	파이썬으로만
사람이 읽기	가능 (글자로 보임)	불가 (알 수 없는 기호)
주 용도	데이터 주고받기	학습한 모델 보관

→ 데이터는 csv로, 학습 결과(모델)는 pkl로 저장한다

Olist 실습에서 이미 만든 파일

model 폴더 안을 들여다보면

내 드라이브 › Colab Notebooks › Data_Analysis › Olist › model

파일 이름	무엇이 들어 있나	크기
part6_recommend.pkl	유사 상품 추천 (KNN)	약 150KB
part7_1_sentiment_ml.pkl	감성 분석 모델 + 단어집	약 170KB
part7_2_tokenizer.pkl	딥러닝용 단어집	463KB
part8_seller_segment.pkl	판매자 유형 분류 (K-Means)	약 3KB
part9_seller_churn.pkl	판매자 이탈 예측	약 2KB
part7_2_sentiment_model.keras	딥러닝 모델 (Keras 전용 형식)	2,541KB

딥러닝 모델은 pkl 대신 Keras 전용 형식(.keras)으로 저장 · 저장하는 원리는 같음

코드로 보기

저장은 dump, 불러오기는 load

① 학습 노트북에서 저장

```
import joblib

# 모델과 단어집을 한 묶음으로
bundle = {"model": model,
          "tfidf": tfidf}

joblib.dump(bundle,
  "part7_1_sentiment_ml.pkl")
```

② 다른 노트북 · 웹앱에서 불러오기

```
import joblib

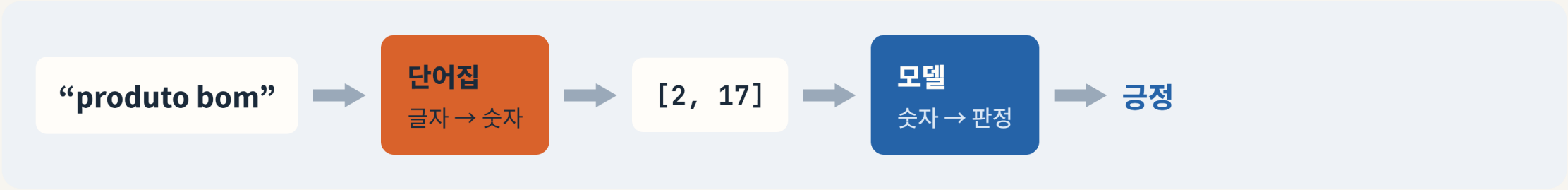
ml = joblib.load(
  "part7_1_sentiment_ml.pkl")

# 학습 없이 바로 예측
X = ml["tfidf"].transform(["produto bom"])
ml["model"].predict(X)
```

joblib = 모델처럼 큰 파이썬 물건을 pkl로 저장할 때 많이 쓰는 도구 (파이썬 기본 pickle과 같은 역할)

왜 단어집도 함께 저장할까?

모델은 글자를 모르고, 번호만 안다



모델만 저장하면

새 리뷰를 숫자로 바꿀 방법이 없음. 단어집을 새로 만들면 같은 단어가 다른 번호가 되어 엉뚱한 예측

함께 저장하면

학습 때와 똑같은 번호로 바꿔 넣을 수 있어, 학습 때와 같은 실력이 나옴

[2, 17]은 이해를 돕기 위한 예시 번호

pkl 파일별 설명

model 폴더의 다섯 파일, 한눈에 보기

파일	하는 일	입력	출력	웹앱에 필요한 것
part6_recommend	비슷한 상품 추천	상품 선택	비슷한 상품 5개	pkl 하나로 충분
part7_1_sentiment_ml	리뷰 긍정/부정	리뷰 문장 입력	긍정 확률	pkl 하나로 충분
part7_2_tokenizer	리뷰 긍정/부정 (딥러닝)	리뷰 문장 입력	긍정 확률	pkl + .keras (비교용)
part8_seller_segment	판매자 유형 분류	판매자 숫자 5개 입력	4가지 유형	pkl만
part9_seller_churn	판매자 이탈 예측	판매자 표에서 선택	이탈 확률	pkl + 판매자 표(csv)

→ 입력이 적으면 pkl만으로 충분하고, 입력 숫자가 14개나 되는 09는 미리 계산한 판매자 표(csv)가 하나 더 필요하다

pkl 파일 설명 1 / 5 · 유사 상품 추천

part6_recommend.pkl

이 상품과 비슷한 상품 5개를 찾아 주는 추천 모델 · 159KB

안에 든 것

이름	뜻
products	상품 2,498개 목록과 특징 (가격·무게·크기 등)
knn_by_category	카테고리 15개별 KNN 모델
scaler	숫자 단위를 맞추는 도구
features · k	사용한 특징 7개 · 추천 개수 5

입력 → 출력

상품 1개 선택 → 비슷한 상품 5개

웹앱: pkl 하나로 충분
상품 목록이 pkl 안에 함께 들어 있음

```
art = joblib.load("part6_recommend.pkl")
recommend("상품 ID")  # 06 노트북 셀 16의 함수 → 비슷한 상품 ID 5개
```

pkl 파일 설명 2 / 5 · 리뷰 감성 분석 (배포용)

part7_1_sentiment_ml.pkl

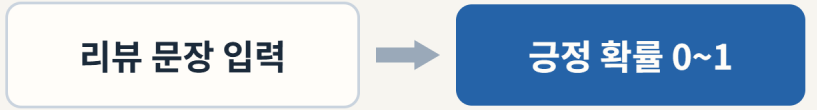
리뷰 문장을 읽고 긍정/부정을 판정하는 머신러닝 모델 · 176KB

안에 든 것

이름	뜻
tfidf	단어집: 문장을 숫자로 바꾸는 도구 (단어 1만 개)
model	로지스틱 회귀: 숫자를 보고 긍정 확률 계산

정확도 0.917 · 가볍고 판단 근거(단어별 가중치)를 볼 수 있어 웹앱에 쓰는 감성 모델

입력 → 출력



웹앱: pkl 하나로 충분
재료(문장)는 사용자가 직접 입력

```
ml = joblib.load("part7_1_sentiment_ml.pkl")
X = ml["tfidf"].transform(["produto veio quebrado, péssimo"])
ml["model"].predict_proba(X)[0, 1]  # → 0.006 (부정) · 상품이 깨져서 왔다, 최악
```

pkl 파일 설명 3 / 5 · 리뷰 감성 분석 (딥러닝, 비교용)

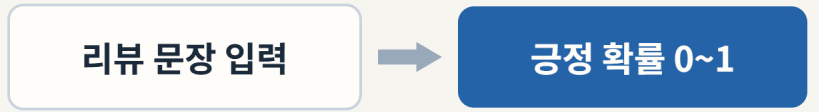
part7_2_tokenizer.pkl

딥러닝 모델용 단어집 · 463KB · 모델 본체는 part7_2_sentiment_model.keras (2,541KB)

세 가지가 한 세트

구성	뜻
tokenizer.pkl	단어 → 번호 (단어 1만 개)
sentiment_model.keras	번호를 읽고 긍정 확률 계산
MAX_LEN = 50	문장 길이. 파일에 없음, 꼭 기억

입력 → 출력



웹앱에는 7_1을 사용
정확도는 비슷(0.924)한데 15배 무거움

```
tokenizer = pickle.load(open("part7_2_tokenizer.pkl", "rb"))
model = load_model("part7_2_sentiment_model.keras")
seq = pad_sequences(tokenizer.texts_to_sequences([문장]), maxlen=50, padding="post")
model.predict(seq)  # → 긍정 확률
```

pkl 파일 설명 4 / 5 · 판매자 유형 분류

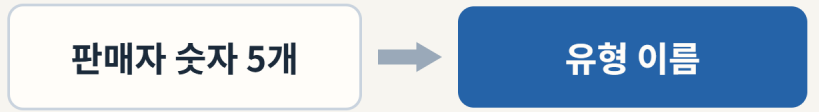
part8_seller_segment.pkl

판매자를 4가지 유형으로 나누는 K-Means 모델 · 3KB

안에 든 것

이름	뜻
kmeans · scaler	유형 4개의 중심점 · 단위 맞추는 도구
names	핵심 우량 · 신규 성장 · 휴면 · 저평점 위험
features	판단에 쓰는 판매자 숫자 5개
review_fill · cutoff	리뷰 없을 때 채울 값 · 기준일 2018-06-01

입력 → 출력



웹앱: 숫자 5개를 직접 입력해 판정
주문 건수 · 판매액 · 평균 별점 · 마지막/첫 판매 후 일수

```
X = make_segment_input(판매자표, art["review_fill"])[art["features"]]
seg = art["kmeans"].predict(art["scaler"].transform(X))
art["names"][seg[0]]  # → 예: "신규 성장" · 08 노트북 셀 15
```

pkl 파일 설명 5 / 5 · 판매자 이탈 예측

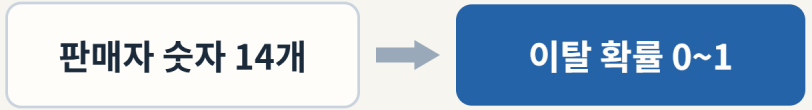
part9_seller_churn.pkl

앞으로 3개월 동안 주문을 못 받을 판매자를 예측하는 모델 · 2KB

안에 든 것

이름	뜻
model	빈칸 채우기 + 단위 맞추기 + 로지스틱 회귀 한 묶음
features	판단에 쓰는 판매자 숫자 14개
threshold	0.5 이상이면 '이탈 위험'
horizon · cutoff	예측 기간 3개월 · 기준일 2018-06-01

입력 → 출력

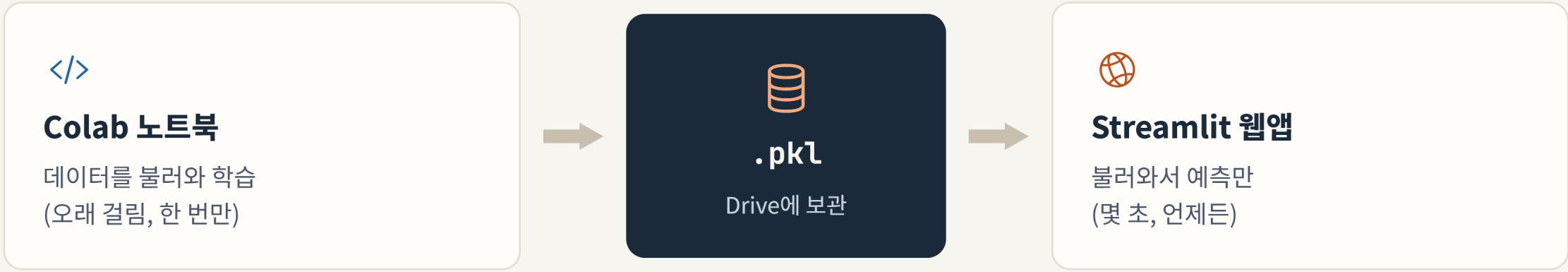


웹앱: 판매자 표(csv)에서 골라 판정
숫자 14개는 직접 입력하기 어려움

```
art = joblib.load("part9_seller_churn.pkl")
prob = art["model"].predict_proba(판매자표[art["features"]])[:, 1]
prob ⩾ art["threshold"]  # True = 이탈 위험 · 09 노트북 셀 18
```

pkl은 어디에 쓰일까?

학습하는 곳과 쓰는 곳을 이어 주는 다리



Streamlit = 파이썬 코드 몇 줄로 웹 화면을 만드는 도구 · 노트북의 예측 코드를 거의 그대로 옮긴다

Streamlit 기본 명령

st.◯◯ 한 줄 = 화면 부품 하나

구분	명령	화면에 생기는 것
글자	st.title · st.write	제목 · 문장
입력	st.text_area	글 상자 (리뷰 입력)
	st.selectbox	골라 쓰는 목록 (상품 · 판매자)
	st.number_input	숫자 칸 (판매자 숫자 5개)
	st.button	버튼 (누르면 True)
결과	st.metric · st.progress	큰 숫자 · 확률 막대
	st.dataframe · st.plotly_chart	표 · 그래프
배치	st.columns · st.tabs · st.sidebar	여러 칸 · 탭 · 왼쪽 막대
메뉴	st.navigation	왼쪽 페이지 메뉴 (app.py)
속도	@st.cache_resource	pkl을 처음 한 번만 불러오기

Streamlit이 움직이는 방식

버튼을 누르거나 값을 바꿀 때마다
코드를 위에서 아래로 다시 실행
→ 화면이 새로 그려짐

그래서 무거운 pkl은 **@st.cache_resource**로
한 번만 불러 두고 계속 재사용

```
python -m streamlit run app.py
```

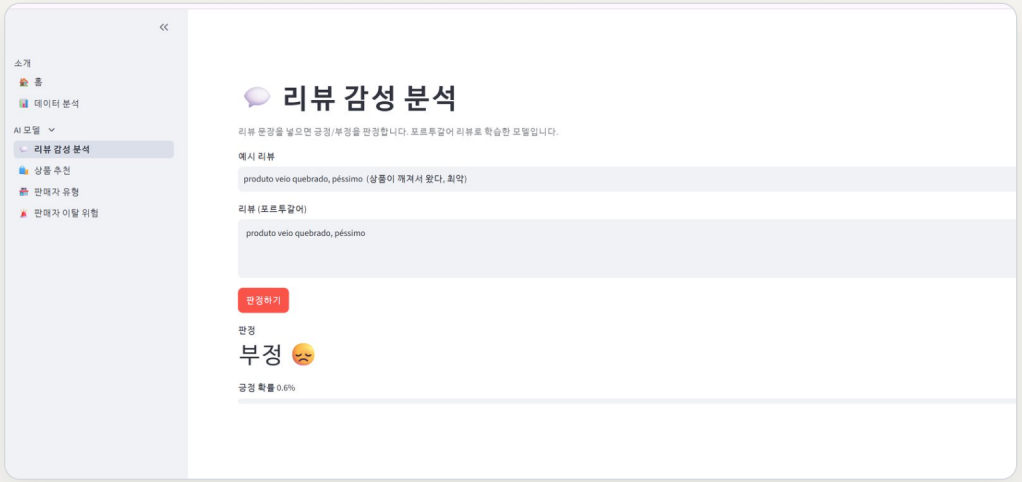
Streamlit으로 이어가기 ① 리뷰 판정 화면

노트북의 예측 코드에 입력 칸과 버튼만 더한다

app.py

```
import streamlit as st
import joblib
ml = joblib.load("part7_1_sentiment_ml.pkl")

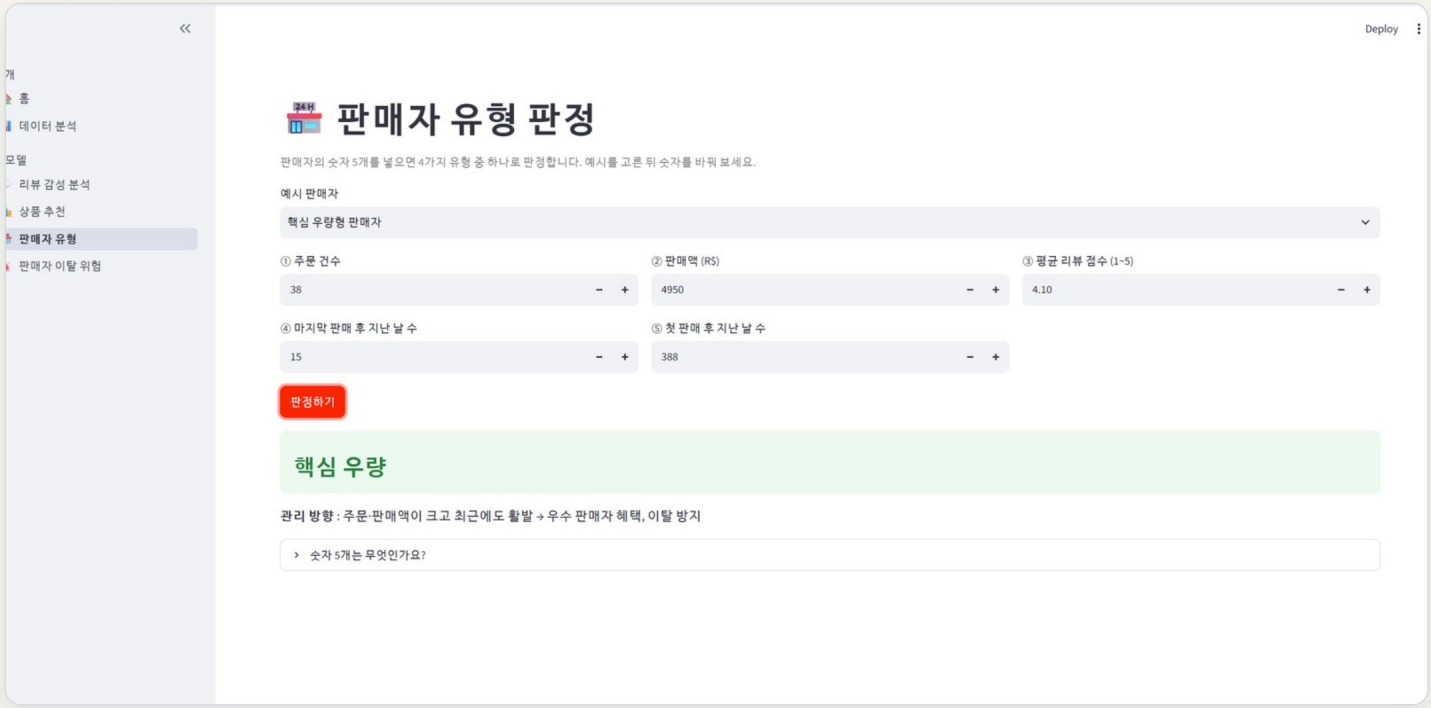
st.title("리뷰 감성 분석")
text = st.text_area("리뷰 입력")
if st.button("판정하기"):
    X = ml["tfidf"].transform([text])
    p = ml["model"].predict_proba(X)[0, 1]
    st.write("긍정" if p ≥ 0.5 else "부정", f"{p:.1%}")
```



주황 글씨(st.◯◯)가 Streamlit 부분 · 나머지는 7_1 노트북의 예측 코드 그대로

Streamlit으로 이어가기 ② 판매자 유형 화면

숫자 5개를 넣으면 유형이 나온다



기능 테스트 순서

1. 예시 판매자 4종을 하나씩 골라 판정 → 이름대로 나오는지 확인

2. 숫자 하나만 바꿔 보기
예: 신규 성장에서 ③ 평균 리뷰를 1.5로 → 저평점 위험

3. ④ 마지막 판매 후 · ⑤ 첫 판매 후를 둘 다 300일로 → 휴면

Streamlit으로 이어가기 ③ 판매자 이탈 화면 준비

이탈 모델은 숫자 14개 → 표를 미리 만들어 둔다

</>

10 노트북

원본 주문 · 배송 · 리뷰 기록으로
판매자 1,817명의 숫자 계산

seller_table.csv

판매자 1명 = 1행

웹앱

표 + part9 pkl
→ 판매자마다 이탈 확률

왜 직접 입력하지 않나?

배송 지연율, 출고 지연율, 1~2점 리뷰 비율, 최근 3개월 · 그 전 3개월 주문 수…
사람이 외워서 넣을 수 없는 숫자가 대부분

```
# 10 노트북 셀 9 아래에 추가
cols = ["seller_id", "frequency", "monetary",
  "orders_last_3m", "orders_prev_3m",
  "sales_last_3m"] + churn["features"]
latest[cols].to_csv(f"{model_path}/seller_table.csv",
  index=False)
```

Streamlit으로 이어가기 ④ 판매자 이탈 화면

순위표에서 판매자를 골라 자세히 본다



화면 읽는 법

예상 손실 = 이탈 확률 × 최근 3개월 판매액
확률과 금액을 함께 보고 순서를 정함

2위는 확률 0.33으로 낮지만 판매액이 커서 위에 있음

기능 테스트

1. 1,817명 · 637명 · 1위 b37c4c02가 10 노트북 결과와 같은지 확인
2. 유형 버튼을 바꿔 순위표 비교
3. 판매자를 골라 지연율 · 리뷰 확인

Streamlit으로 이어가기 ⑤ 완성 그림

웹앱 하나에 AI 화면 네 개

리뷰 감성 분석

문장 입력 → 긍정/부정과 확률

part7_1 pkl

상품 추천

카테고리 · 상품 선택 → 비슷한 상품 5개

part6 pkl

판매자 유형

숫자 5개 입력 → 4가지 유형 중 하나

part8 pkl

판매자 이탈 위험

순위표 · 판매자 선택 → 이탈 확률

part9 pkl + 판매자 표

```
olist_dashboard/
├─ app.py            # 왼쪽 메뉴
├─ views/            # 화면 6개
├─ utils/models.py # 예측 함수
├─ model/            # pkl 4개 (약 340KB)
├─ data/             # 주문 · 판매자 표
└─ requirements.txt
```

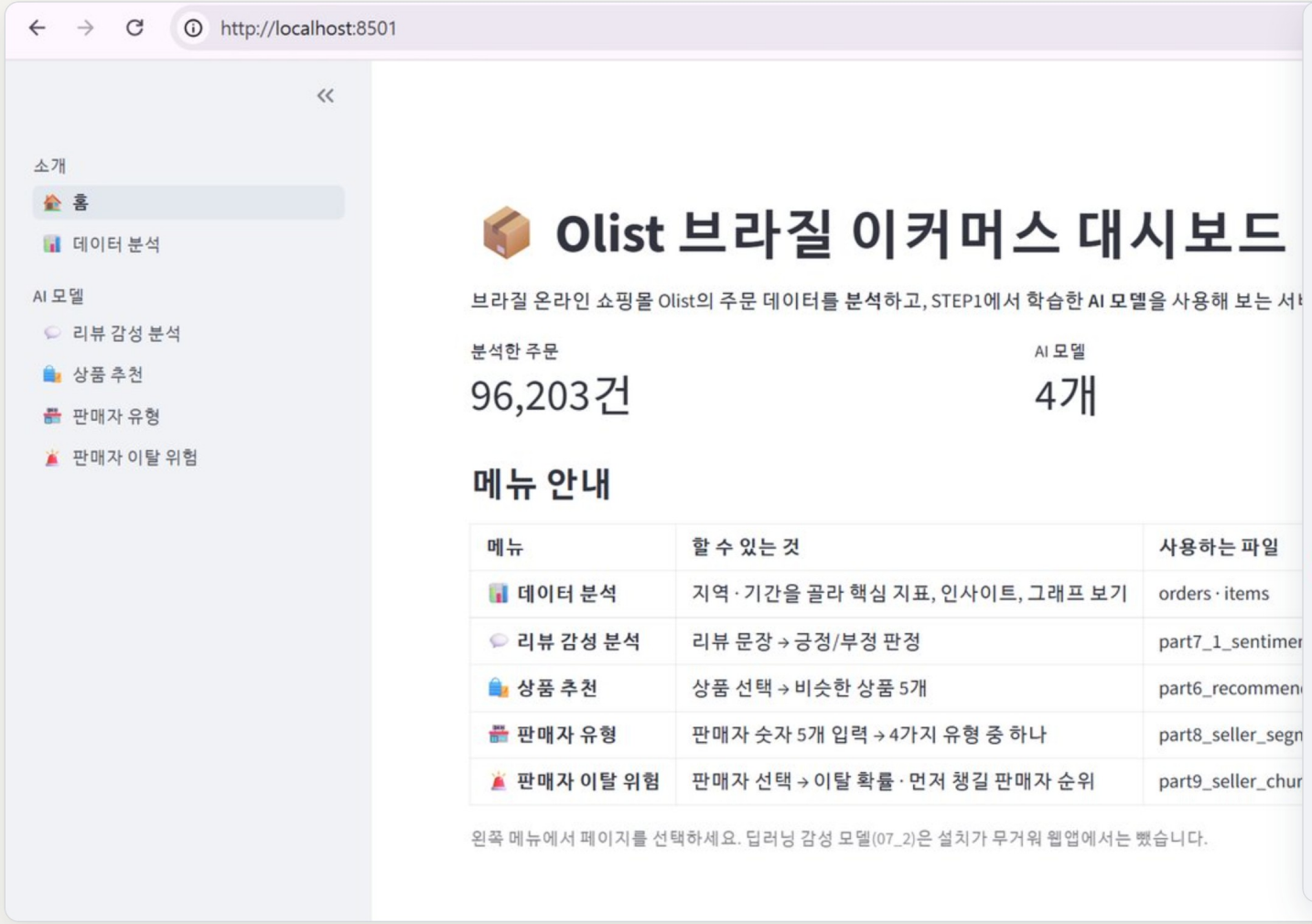
requirements.txt가 핵심

scikit-learn · numpy를 학습한 Colab과 같은 버전으로 고정
버전이 다르면 pkl이 열리지 않거나 결과가 달라질 수 있음

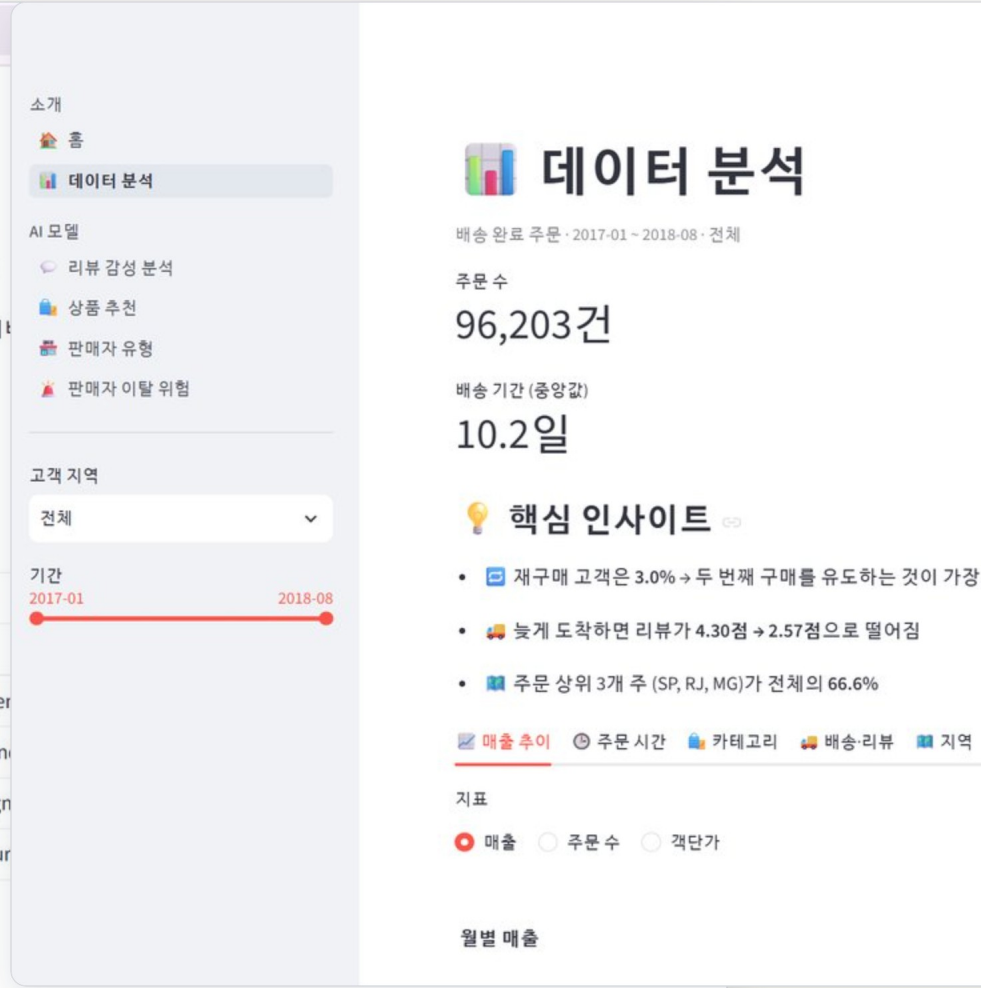
딥러닝(07_2)은 TensorFlow 설치가 수백 MB라 웹앱에서 제외 · Colab 데모에서만 비교

Streamlit으로 이어가기 · 실제 실행 화면

http://localhost:8501 에서 열린 대시보드



🏠 **홈** · 왼쪽 메뉴: 소개 2개 + AI 모델 4개, 메뉴마다 쓰는 pkl 파일 안내



📊 **데이터 분석** · 지역 · 기간을 바꾸면 지표 · 문장 · 그래프가 함께 바뀜

Streamlit 실행 순서

Colab 2번 → 폴더 맞추기 → VS Code 실행

① Colab

판매자 표 저장

10 노트북 셀 9 아래에 저장 셀 추가 → 실행

`seller_table.csv`

② Colab

배포 파일 받기

00_prepare_deploy_files 끝까지 실행

`olist_deploy_files.zip`

③ 내 컴퓨터

폴더 맞추기

olist_dashboard 안에 zip의 data · model · requirements.txt 넣기

VS Code → 폴더 열기

④ VS Code 터미널

설치하고 실행

Python 3.12 가상환경 → 설치 → 실행

브라우저 localhost:8501

```
olist_dashboard/
├─ app.py · views/ · utils/
├─ data/  # orders · items · seller_table
├─ model/ # pkl 4개
└─ requirements.txt
```

끝낼 때 · 다시 열 때

끝내기: 터미널에서 Ctrl + C
다시 열기: .venv\Scripts\activate → python -m streamlit run app.py

Streamlit으로 이어가기 ⑥ 내 컴퓨터에서 실행하기

Python 3.12 가상환경에서 실행한다

Python 버전별 결과

버전	결과	이유
3.12	✅ 추천	모델을 학습한 Colab과 같은 버전
3.10 · 3.11	△	numpy 최신 버전이 설치되지 않음 → 버전을 낮춰야 함
3.14	❌	scikit-learn 1.6.1 설치 파일이 없음 → 설치 실패

가상환경 = 이 프로젝트 전용 Python 상자
다른 버전이 깔려 있어도 섞이지 않고, .venv 폴더만 지우면 깨끗이 정리

Streamlit Cloud 배포 때도 Advanced settings에서 **Python 3.12** 선택

VS Code 터미널 (olist_dashboard 폴더)

```
# ① 3.12 설치 확인
py -3.12 --version
# ② 가상환경 만들기 → 들어가기
py -3.12 -m venv .venv
.venv\Scripts\activate
(.venv) 표시 확인
# ③ 설치 → 실행
python -m pip install -r requirements.txt
python -m streamlit run app.py
```

주의할 점

pkl을 쓸 때 꼭 지킬 세 가지

버전 맞추기

저장할 때와 불러올 때 라이브러리(scikit-learn 등) 버전이 다르면 경고나 오류가 날 수 있음

짝꿍도 함께

단어집처럼 입력을 바꾸는 도구도 반드시 같이 저장하고 같이 불러오기

출처 확인

모르는 곳에서 받은 pkl은 열지 않기. 불러오는 순간 안에 숨긴 코드가 실행될 수 있음

→ 내가 만든 pkl, 믿을 수 있는 곳의 pkl만 불러온다

정리

pkl 한 줄 요약

1 **pkl** = 학습한 모델 같은 파이썬 물건을 통째로 저장한 파일

2 **dump로 저장, load로 불러오기** → 다시 학습하지 않고 바로 예측

3 **모델 + 단어집을 함께**, 같은 버전으로, 믿을 수 있는 파일만